

Hands-On: Creating Class and Function Components in React

Prerequisites

To complete this lab, you will need:

- Node.js and NPM installed on your computer
- A code editor, such as Visual Studio Code

Setup

1. Create a new directory for your project and navigate into it using your terminal or command prompt:

```
mkdir react-components-lab
```

```
cd react-components-lab
```

2. Initialize a new Node.js project:

```
npm init -y
```

3. Install React and ReactDOM:

```
npm install react react-dom
```

4. Install Babel and Webpack:

```
npm install --save-dev @babel/core @babel/preset-env @babel/preset-react babel-loader  
webpack webpack-cli webpack-dev-server
```

Building your first class component

1. Create a new file called **ClassComponent.js** in your project directory and add the following code:

```
import React from 'react';  
  
class ClassComponent extends React.Component {  
  render() {  
    return <h1>Hello, Class Component!</h1>;  
  }  
}  
  
export default ClassComponent;
```

2. Create a new file called **index.js** in your project directory and add the following code:

```
import React from 'react';  
import ReactDOM from 'react-dom';  
import ClassComponent from './ClassComponent';  
  
ReactDOM.render(<ClassComponent />, document.getElementById('root'));
```

3. Create a new file called **index.html** in your project directory and add the following code:

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>React Components Lab</title>
  </head>
  <body>
    <div id="root"></div>
    <script src="bundle.js"></script>
  </body>
</html>
```

4. Create a new file called **webpack.config.js** in your project directory and add the following code:

```
module.exports = {
  entry: './index.js',
  output: {
    path: __dirname,
    filename: 'bundle.js',
  },
  module: {
    rules: [
      {
        test: /\.jsx?$/,
        exclude: /node_modules/,
        use: {
          loader: 'babel-loader',
          options: {
            presets: ['@babel/preset-env', '@babel/preset-react'],
          },
        },
      },
    ],
  },
  devServer: {
    contentBase: './',
    port: 8080,
  },
};
```

5. Build and run the project:

```
npx webpack-dev-server
```

6. Open your browser and go to **http://localhost:8080**. You should see the text "Hello, Class Component!" displayed on the page.

Building your first function component

1. Create a new file called **FunctionComponent.js** in your project directory and add the following code:

```
import React from 'react';

const FunctionComponent = () => {
  return <h1>Hello, Function Component!</h1>;
};

export default FunctionComponent;
```

2. Modify **index.js** to import and use the **FunctionComponent** component:

```
import React from 'react';
import ReactDOM from 'react-dom';
import ClassComponent from './ClassComponent';
import FunctionComponent from './FunctionComponent';

ReactDOM.render(
  <div>
    <ClassComponent />
    <FunctionComponent />
  </div>,
  document.getElementById('root')
);
```

3. Build and run the project:

```
npx webpack-dev-server
```

4. Open your browser and go to **http://localhost:8080**.

You should now see both the "Hello, Class Component!" and "Hello, Function Component!" texts displayed on the page.

Conclusion

In this hands-on lab, you learned how to create class components and function components in React. You also learned how to use Babel and Webpack to compile your code and run a local development server.

Both class components and function components have their own advantages and use cases in React development. Class components are used when you need to manage state or use lifecycle methods, while function components are used for simpler components without state or lifecycle methods.

Keep practicing and building with React components, and you'll soon become proficient in React development.